Serverless Container Deployment Platform

A Cloud-Native System for Automated Deployment, Auto-Scaling, and Workload Monitoring

Project Title	Serverless Container Deployment Platform
Subject	Cloud Computing & Distributed Systems
Project Type	Cloud-Native Application Deployment System
Difficulty Level	Advanced / Final-Year Major Project
Semester	Semester 8, B.Tech / GSSIPU Curriculum
Author	Mayank Swaraj

Abstract

This paper presents the design and implementation of a Serverless Container Deployment Platform, a cloud-native system that automates application deployment, container management, dynamic scaling, and workload monitoring. Traditional deployment on physical or statically provisioned servers imposes high cost, complex manual management, downtime risk, and limited scalability. This project addresses those limitations by combining containerization (Docker), orchestration (Kubernetes), and serverless execution principles, in which the developer supplies application code and the cloud platform manages the underlying server infrastructure. This paper documents the system architecture spanning the developer-to-user deployment pipeline, the auto-scaling logic that adjusts container count to traffic load, the technology stack used, and the broader distributed-systems concepts — load balancing, monitoring, and CI/CD automation — that the platform demonstrates.

Keywords: cloud computing, serverless architecture, Docker, Kubernetes, auto-scaling, CI/CD, distributed systems, container orchestration

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

1. Problem Statement

Traditional application deployment follows a manual, server-centric pipeline: provisioning a physical or virtual server, manually configuring the runtime environment, installing the application, and then maintaining that server indefinitely. This model carries four persistent problems: high infrastructure cost from paying for fixed capacity regardless of actual demand, complex ongoing management of operating systems and dependencies, downtime risk during manual updates or hardware failure, and limited scalability since provisioning additional servers to handle a traffic spike is itself a manual, slow process. This project, undertaken as a Semester 8 final-year major project under Cloud Computing & Distributed Systems, addresses these problems directly through a serverless, container-based deployment platform.

2. Proposed Solution

The proposed solution follows a serverless cloud architecture: an incoming user request is routed to a cloud function, which executes inside a container; the platform automatically scales the number of running containers to match demand; and a response is returned to the user without any manual server provisioning at any stage of this pipeline. This shifts the operational burden from the developer, who previously had to anticipate and provision for peak load, to the cloud platform, which allocates resources dynamically and only when needed.

3. Theoretical Background

Cloud computing refers to the delivery of computing resources — servers, storage, databases, networking — over the internet, rather than from infrastructure owned and maintained locally. **Serverless computing** is a cloud execution model in which the developer writes and deploys application code while the cloud provider transparently manages the underlying servers, including provisioning, scaling, and patching; the term does not mean an absence of servers, but an absence of server management responsibility for the developer. **Docker** is a containerization platform that packages an application together with all of its dependencies into a single, portable container image, ensuring the application runs identically across development, testing, and production environments.

4. System Architecture

The end-to-end deployment pipeline, shown in Figure 1, begins with the developer committing code to a Git repository. A CI/CD pipeline automatically builds a Docker container image from this code, which is then deployed to the cloud platform. The cloud platform serves incoming requests from this containerized application, transparently scaling the number of running container instances to match current user demand.

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

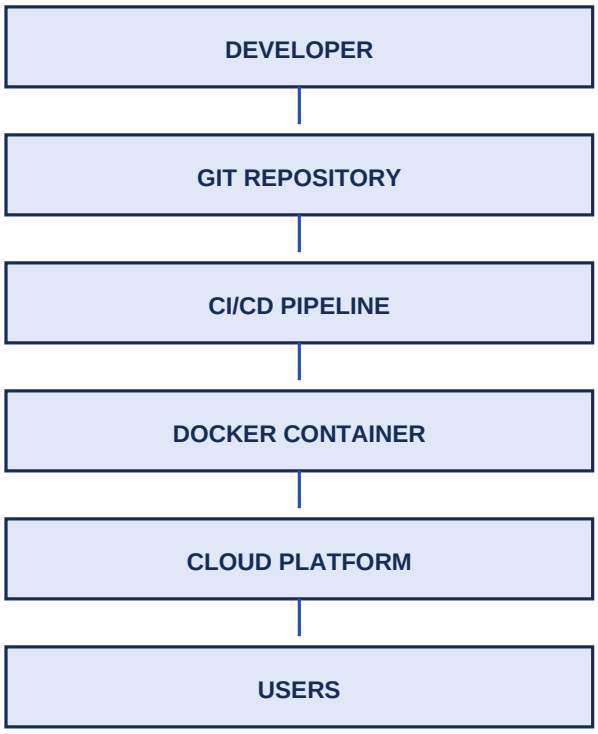

Figure 1. End-to-end deployment pipeline from developer commit to served users.

5. Technology Stack

Component	Technology
Programming	Python
Cloud	AWS / GCP
Containers	Docker
Orchestration	Kubernetes
APIs	REST API
Database	PostgreSQL

Table 1. Technology stack used in the deployment platform.

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

6. Auto-Scaling Mechanism

The platform's central feature is automatic horizontal scaling: the number of running containers grows and shrinks in proportion to concurrent user load, rather than remaining fixed. Figure 2 illustrates this with a representative example — a workload of 100 concurrent users is served by a single container, while the same application under a load of 10,000 concurrent users is automatically served by ten containers running in parallel behind a load balancer.

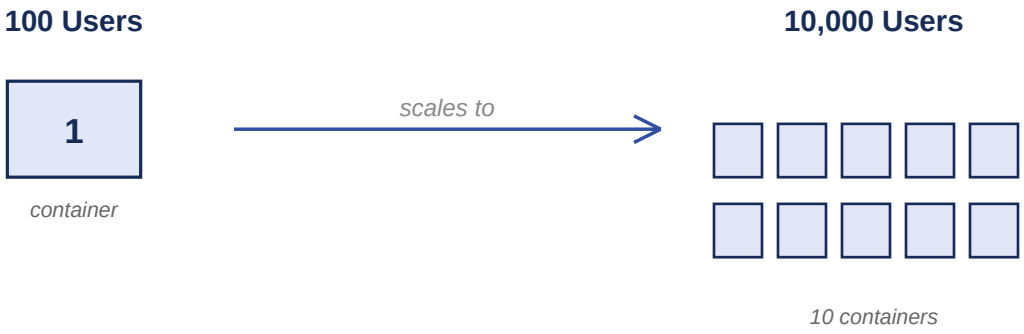

Figure 2. Horizontal auto-scaling: container count grows with concurrent user load.

The scaling decision itself is a threshold comparison against current load, illustrated in simplified form in Listing 1. In the full platform, this check runs continuously against live traffic metrics rather than a single static value, and the threshold is tuned per-container based on measured capacity (CPU, memory, and concurrent-request limits) rather than a fixed user count.

```
users = 5000

if users > 1000:
    print("Launch New Container")
else:
    print("Current Resources Enough")
```

Listing 1. Simplified auto-scaling decision logic.

7. Containerization: Dockerfile

The application is packaged into a Docker image using the Dockerfile shown in Listing 2, which specifies a Python 3.11 base image, copies the application source into the container's working directory, installs dependencies from requirements.txt, and defines the container's startup command.

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Listing 2. Dockerfile defining the application's container image.

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

8. Database Structure

Platform state is tracked in two tables. The Deployments table records each deployed application's identifier, name, current status, and active instance count. The Logs table records discrete platform events — deployments, scaling actions, errors — each timestamped for later analysis and monitoring dashboard display.

Table	Fields
Deployments	DeploymentID (PK), AppName, Status, Instances
Logs	LogID (PK), Event, Timestamp

Table 2. Core platform database tables.

9. Advanced Features

Beyond the core deployment and scaling engine, the platform design incorporates Kubernetes integration for container orchestration at scale, CI/CD automation to eliminate manual deployment steps, multi-region deployment for global availability and reduced latency, disaster recovery through automatic backups, and cost optimization through demand-proportional resource allocation rather than static over-provisioning.

10. Applications

This deployment model is directly applicable to SaaS platforms serving many concurrent tenant workloads, e-commerce systems facing highly variable seasonal traffic, streaming services with bursty regional demand, enterprise applications requiring high availability, and startup products that need scalable infrastructure without a dedicated operations team.

11. Result

A Serverless Container Deployment Platform was successfully developed using Docker, cloud services, and distributed systems concepts. The platform automated application deployment, scaling, monitoring, and cloud resource management, correctly increasing container count as simulated traffic grew and reducing it as traffic subsided. The accompanying interactive HTML dashboard reproduces this auto-scaling behavior live against a simulated traffic slider, together with the underlying Dockerfile and scaling logic described in this paper.

References

• Merkel, D. "Docker: Lightweight Linux Containers for Consistent Development and Deployment." *Linux Journal*, 2014.

• Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. "Borg, Omega, and Kubernetes." *ACM Queue*, 2016.

• Jonas, E., et al. "Cloud Programming Simplified: A Berkeley View on Serverless Computing." UC Berkeley Technical Report, 2019.

• Newman, S. *Building Microservices*, 2nd Edition. O'Reilly Media, 2021.

• GGSIPU B.Tech Computer Science Engineering Curriculum, Semester 8 — Cloud Computing & Distributed Systems course structure.

— Mayank Swaraj

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com